```
puts "Shipping 16oz domestic"
puts Shipping.shipping_options(16, false)

puts "\nShipping 90oz domestic"
puts Shipping.shipping_options(90, false)

puts "\nShipping 16oz international"
puts Shipping.shipping_options(16, true)
```

produces:

```
Shipping 16oz domestic
MediaMail
FlatRatePriorityEnvelope

Shipping 90oz domestic
MediaMail

Shipping 16oz international
InternationalFlatRateBox
```

Command interpreters often use this pattern: the base class keeps a track of available commands, each of which is implemented in a subclass.

The method_missing Hook

Earlier, we saw how Ruby executes a method call by looking for the method, first in the object's class, then in its superclass, then in that class's superclass, and so on. If the method call has an explicit receiver, then private methods are skipped in this search. If the method is not found by the time we run out of superclasses (because BasicObject has no superclass), then Ruby tries to invoke the hook method method_missing on the original object. Again, the same process is followed—Ruby first looks in the object's class, then in its superclass, and so on. However, Ruby predefines its own version of method_missing in class BasicObject, so typically the search stops there. The built-in method_missing basically raises an exception (either a NoMethodError or a NameError depending on the circumstances).

The key here is that method_missing is simply a Ruby method. We can override it in our own classes to handle calls to otherwise undefined methods in an application-specific way.

The method_missing method has a signature that includes the name of the method being sought and the arguments:

```
def method_missing(name, *args, &block)  # ...
```

The name argument receives the name of the method that couldn't be found. It is passed as a symbol. The args argument is an array of the arguments that were passed in the original call. And the oft-forgotten block argument will receive any block passed to the original method.

```
def method_missing(name, *args, &block)
  puts "Called #{name} with #{args.inspect} and #{block}"
end

wibble
wobble 1, 2
wurble(3, 4) { stuff }
```